System Architecture

Foundation document | Version 0.1 | 07 August 2026

Architecture summary

STF is a modular, multi-tenant SaaS with a mobile-first web application and an administrative web experience. Start as a well-structured modular monolith with clear boundaries; split services only when scale, reliability, or team ownership proves the need.

Logical layers

Client layer: responsive web/PWA, accessible design system, offline queue for approved attendance cases, and secure session handling. Application layer: identity, authorization, module/flag evaluation, domain modules, approval workflows, and reporting. Data layer: relational database with tenant scoping, object storage for documents/proof, audit store, cache/queue, and search only when justified. Integration layer: notification providers and future accounting/biometric connectors behind adapters.

Domain boundaries

Platform: tenants, plans, subscriptions, module catalog, platform admins. Workforce: employees, org units, documents. Attendance and Leave: time events, shifts, exceptions, policies, approvals. Payroll: salary structure, payroll run, calculation inputs, adjustments, payslip. Work: tasks, proof, templates, daily summaries. Shared: identity, permissions, feature flags, notifications, files, audit, reports.

Security boundary

Authenticate users first; resolve tenant and membership second; evaluate role, permission, module, feature, and record scope third. All database queries carry tenant context. Server-generated audit records cover sensitive changes. Store files outside public paths with signed access and tenant validation.

Reliability decisions

Use queued jobs for notifications, report generation, file processing, and scheduled daily summaries. Jobs must be idempotent and traceable. Preserve calculation inputs and policy versions for payroll. Capture offline attendance as pending evidence, then validate it server-side when synchronized.

Data principles

Use stable IDs, timestamps, soft deletion where business recovery requires it, and immutable audit events. Do not store raw secrets in application data. Define retention for location, files, payroll, and logs before production.

Give this to Claude

Propose a technology implementation only after the approved design system and detailed requirements exist. Keep the modular-monolith boundaries above, tenant scoping, flag enforcement, auditability, and queued jobs intact.